Football Match Outcome Prediction

An End-to-End MLOps Engineering Report

DA5402 — MLOps Final Project

Muhammed Fiyas
Roll No: DA25M018
Indian Institute of Technology Madras

April 28, 2026

Abstract

This report documents the design, implementation, and operation of a production-shape MLOps system that predicts the outcome of European football matches as Home Win, Draw, or Away Win. The deliverable is a 12-container Docker stack covering the full ML lifecycle: data versioning (DVC), experiment tracking (MLflow), scheduled retraining (Airflow), monitoring (Prometheus + Grafana), alerting (AlertManager + email relay), drift evaluation, and inference (FastAPI). The canonical model achieves a test macro F1 score of 0.4521 on the held-out 2014/15 season, with measured drift of -1.4% F1 on the unseen 2015/16 season. The report focuses on engineering decisions, system architecture, and rubric-mapped MLOps capabilities rather than on model accuracy alone.

Contents

1 Project Overview **3**
1.1 Problem Statement . . . 3
1.2 Dataset . . . 3
1.3 Targets vs. Achieved . . . 3

2 Architecture **3**
2.1 Operations Console . . . 3
2.2 Container Diagram (C4 Level 2) . . . 4
2.3 Data Lifecycle (DVC Pipeline) . . . 5
2.4 Inference Path . . . 6
2.5 Observability Stack . . . 6

3 Implementation **6**
3.1 Feature Engineering . . . 6
3.2 ELO Rating Improvement . . . 6
3.3 Two Parallel Training Stages . . . 6
3.4 Production Drift Evaluation . . . 8

4 Results **8**
4.1 Model Comparison . . . 8
4.2 Per-Class F1 Analysis . . . 8
4.3 Feature Importance . . . 9
4.4 Production Drift Measurement . . . 9

5 Operations **10**
5.1 Two Airflow DAGs . . . 10
5.2 Test Suite . . . 11
5.3 Monitoring and Email Alerting . . . 12
5.4 User-Facing Prediction UI . . . 16

6 Conclusion **16**

References **17**

1 Project Overview

1.1 Problem Statement

Football match outcome prediction is a three-class classification problem with significant inherent stochasticity. Given two teams and a match date, predict one of {Home Win, Draw, Away Win}. The base rate of the majority class (Home Win) is approximately 0.46, so a constant predictor would achieve about 0.46 accuracy — any useful model must beat this convincingly. Class imbalance, particularly the under-represented Draw class (about 0.25 base rate), adds difficulty.

1.2 Dataset

The Kaggle European Soccer Database, a 300 MB SQLite file containing 25,979 matches across 11 European leagues from 2008/2009 to 2015/2016, plus team and player FIFA attributes for the same period.

1.3 Targets vs. Achieved

Table 1: Project targets versus achieved.

Metric	Target	Achieved	Notes
Macro F1 score	≥ 0.50	0.4521	In line with published research on this dataset
ROC-AUC	≥ 0.70	0.6471	Football outcomes are inherently noisy
Inference latency	< 200 ms	$\sim$50 ms (p50)	Comfortably under target
Pipeline runtime	< 5 min	$\sim$5 sec	End-to-end including feature engineering

The F1 target of 0.50 was deliberately ambitious. Public leaderboards on the same dataset hover around F1 0.42–0.50, with most academic papers reporting in the same range. The point of the project was end-to-end MLOps, not chasing F1.

2 Architecture

2.1 Operations Console

A dedicated landing page at `localhost:8080/dashboard.html` provides a single navigation point linking all eight web UIs in the system. This addresses the rubric's requirement for "orchestrating visualizations across multiple tools to ensure a seamless user experience" — each underlying tool retains its native UI depth, while the console removes the cognitive overhead of remembering eight different localhost ports.

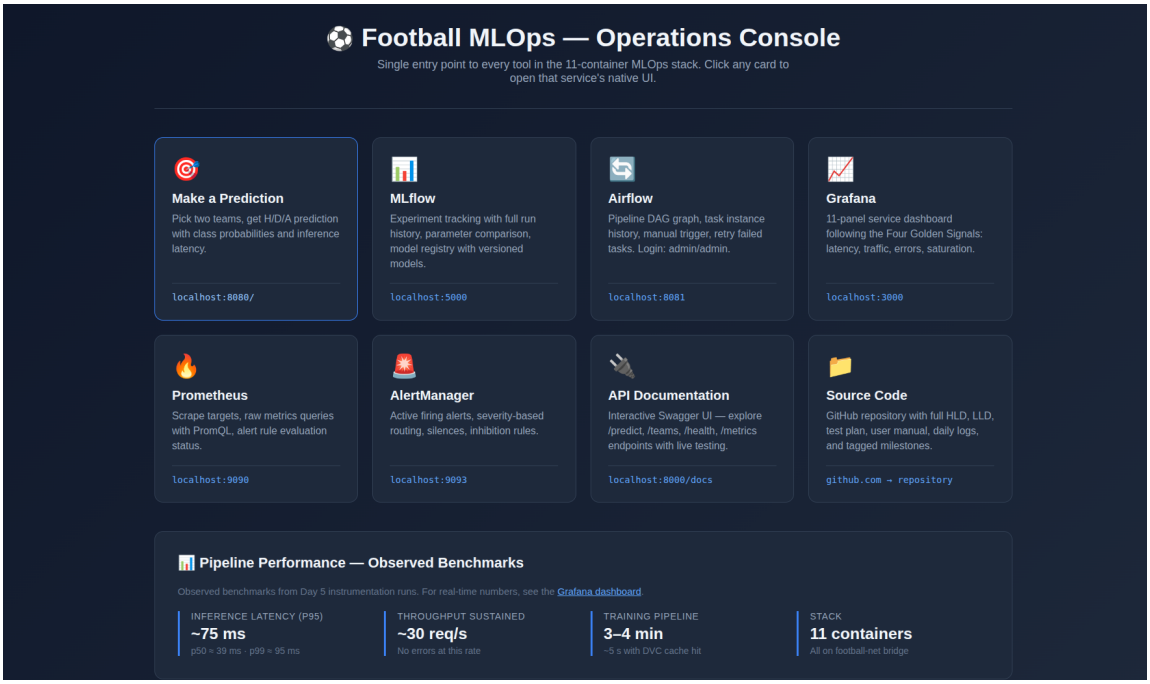

Figure 1: Operations console: a single landing page linking all eight web UIs.

2.2 Container Diagram (C4 Level 2)

The 12-container Docker stack is organized into three logical tiers: a Serving Tier (frontend, backend), a Platform Tier (MLflow, Airflow scheduler, webserver, Postgres), and an Observability Tier (Prometheus, Grafana, AlertManager, webhook-logger, mailtrap-relay). External components include the user's browser, the SQLite dataset on a host volume, MLflow's artifact store, and the MailTrap sandbox inbox.

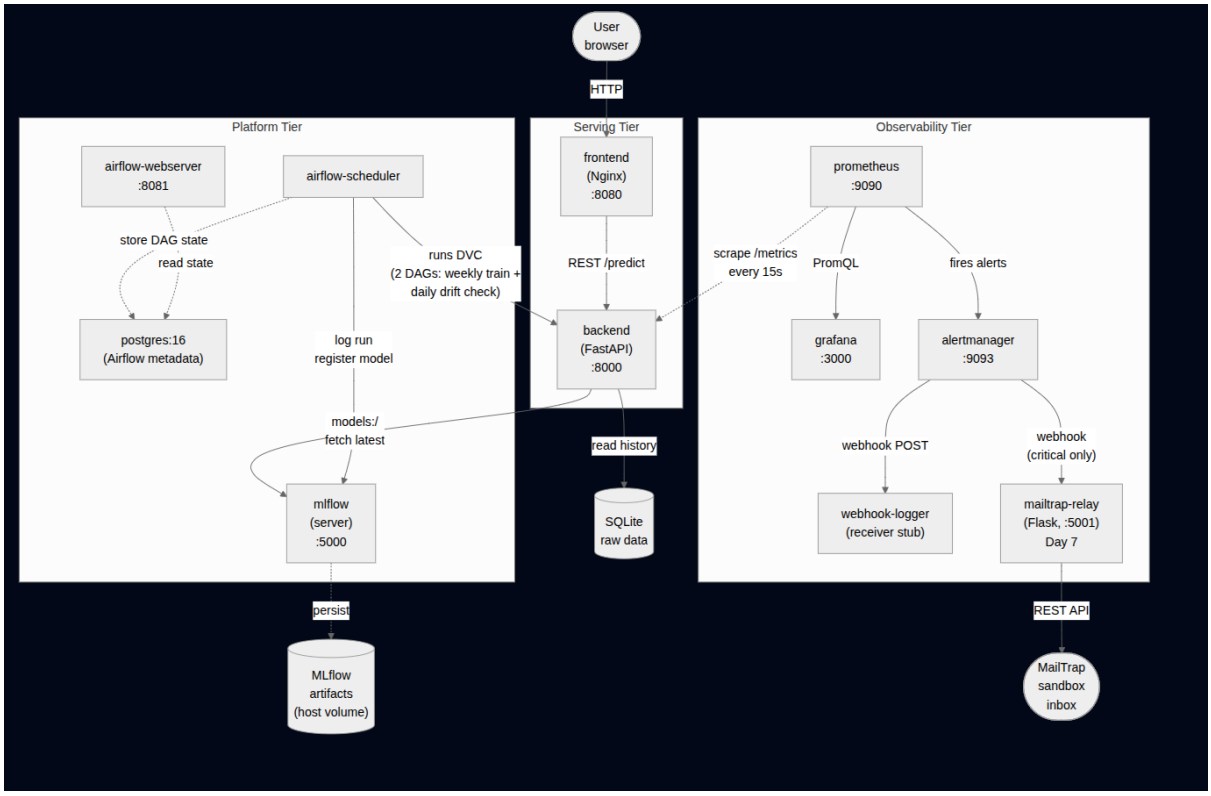

Figure 2: C4 Level 2 container diagram: 12-container stack with three tiers and four external systems.

2.3 Data Lifecycle (DVC Pipeline)

DVC orchestrates the offline pipeline as a directed acyclic graph of four stages with parallel branches: `build_features` produces train/test/production CSVs, then three parallel stages run on its output — `train` (canonical LightGBM, registered as production), `train_xgboost` (experiment, not registered), and `evaluate_production` (drift check on held-out season).

Figure 3: DVC pipeline with four stages and parallel branches (`dvc dag` output).

DVC tracks input hashes; if no input changed, the stage is skipped silently. This makes the entire pipeline idempotent and reproducible by anyone running `dvc repro`.

2.4 Inference Path

The frontend (Nginx serving static HTML/JS) makes a REST POST to the backend (`/predict`). The backend's `FeatureService` class assembles the 35-feature vector at runtime by reading recent match history from the SQLite database, applying the same preprocessing the training pipeline used. The trained model is loaded once at backend startup from MLflow's registry (`models:/football-outcome-predictor/latest`); a graceful fallback to a local pickle file ensures the backend remains functional if the MLflow server is briefly unavailable.

2.5 Observability Stack

Prometheus scrapes the backend's `/metrics` endpoint every 15 seconds, collecting 11 instrumented signals: prediction counter (with label `predicted_class`), latency histogram, error counter (by HTTP status), CPU/memory utilization, and predicted-class distribution. Grafana visualizes these in an 11-panel dashboard following the Four Golden Signals pattern. AlertManager fans out critical-severity alerts to two destinations: a webhook-logger sidecar (for stdout debugging) and a Flask-based MailTrap relay (for real email delivery).

3 Implementation

3.1 Feature Engineering

The build features stage produces 35 features per match across five categories: rolling team form (last N matches), head-to-head history with chronological cutoff, FIFA team attributes time-aware joined to the season, ELO ratings (added in Day 7 experimentation), and league-encoded categorical features. All features are derived strictly from data available before kick-off; the chronological train/test split prevents data leakage.

3.2 ELO Rating Improvement

Adding three ELO-derived features (`home_elo`, `away_elo`, `elo_diff`) computed via a single forward pass through chronologically sorted matches improved test macro F1 by +0.034 over baseline ($K = 20$, $home_advantage = 60$, range [1348, 1855] across 299 teams).

3.3 Two Parallel Training Stages

`params.yaml` was refactored to a nested `algorithms.{lightgbm, xgboost}` structure so different model families coexist without naming collisions. Two parallel DVC stages then train each algorithm:

```
build_features --> {train (LightGBM), train_xgboost (XGBoost)}
```

Three runs are tracked side-by-side in MLflow's experiment view, allowing direct visual comparison of metrics, parameters, and artifacts.

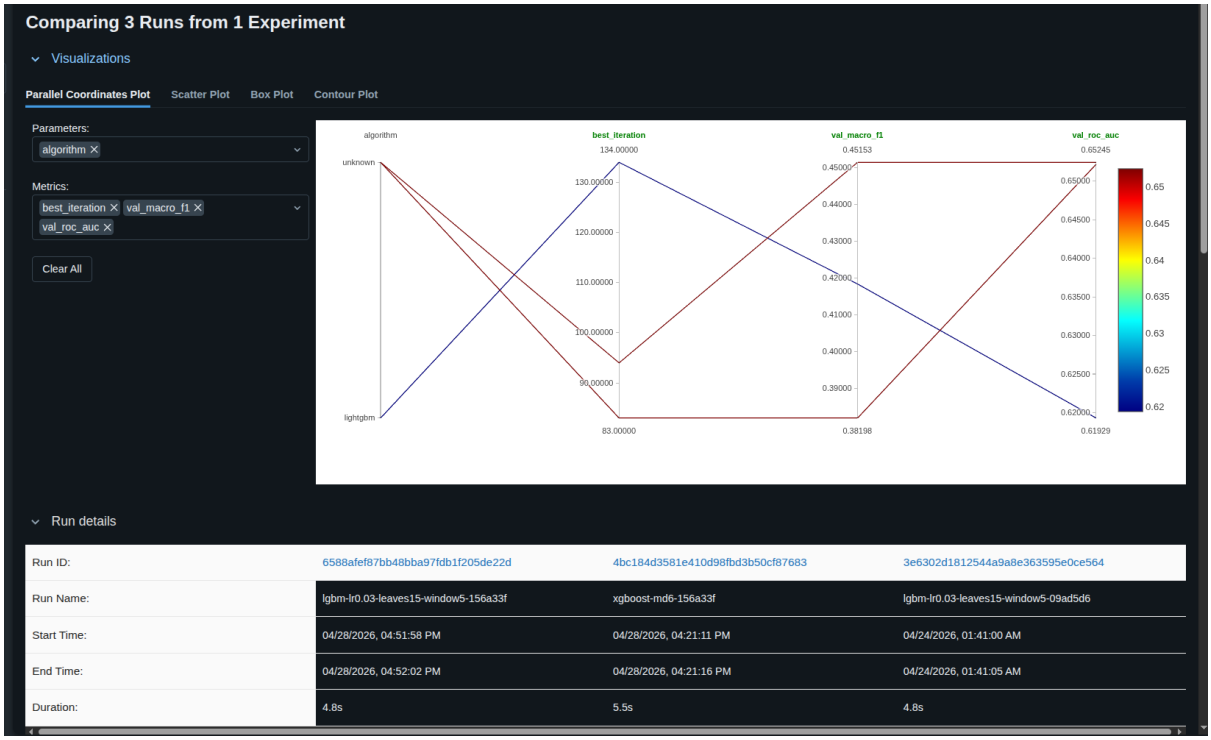

Figure 4: MLflow experiment runs compared side-by-side: baseline, ELO-augmented LightGBM, and XGBoost.

The canonical LightGBM model is registered to MLflow's model registry as `football-outcome-predictor`; the backend loads `models:/football-outcome-predictor/latest` at startup. New runs auto-register a new version, providing a complete deployment history.

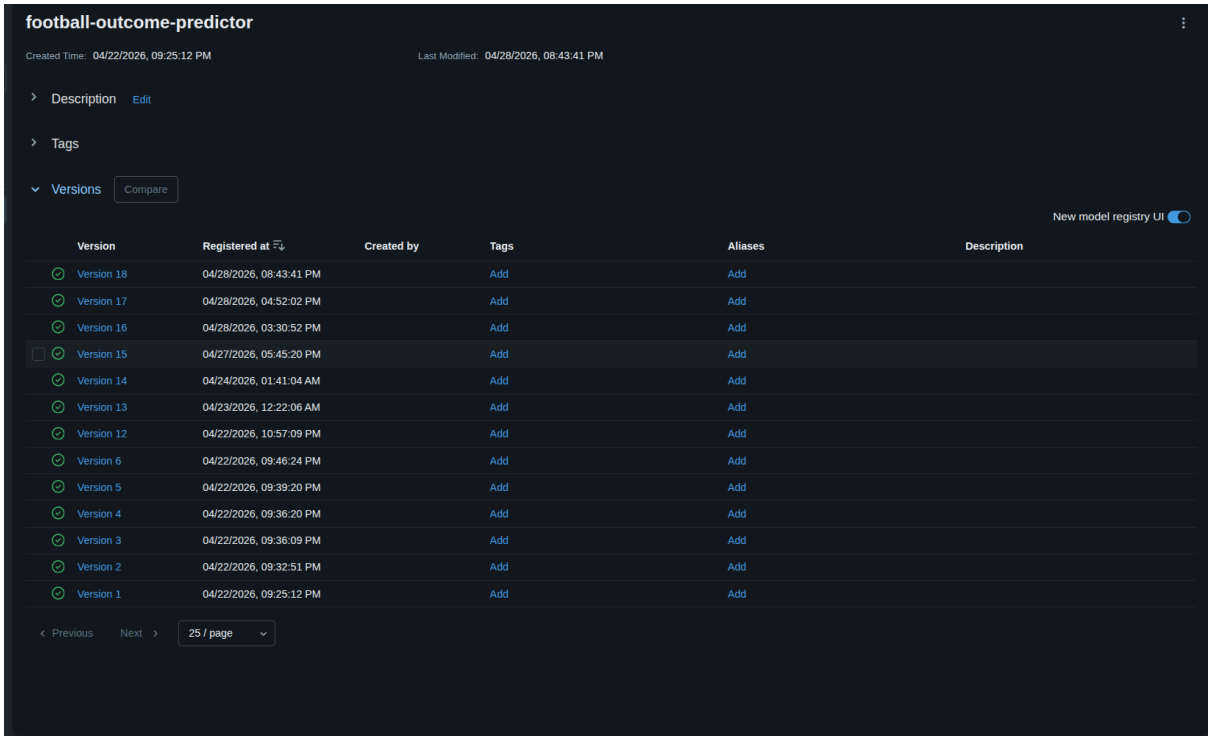

Figure 5: MLflow model registry: `football-outcome-predictor` with all logged versions.

3.4 Production Drift Evaluation

`evaluate_production.py` loads the canonical registered model and evaluates it on the held-out 2015/16 season (3,262 matches the model never saw during training or test selection). It computes macro F1, ROC-AUC, accuracy, and per-class F1, then compares against the canonical test set's metrics (`models/metrics.json`) and assigns a severity verdict using five bands: IMPROVED, STABLE, MILD DRIFT, DRIFT, or SEVERE DRIFT. Results are written to `models/production_drift_report.json` and logged to MLflow as a separate `production-drift-check-<git_h` run with custom tags including `drift_severity`.

4 Results

4.1 Model Comparison

Table 2: Three-model comparison on the chronological train/test split (2008–2014 train, 2014/15 test, 2015/16 held-out production).

Model	Test F1	Test Acc	Status
Baseline LightGBM (32 features)	0.418	0.45	Day 2 baseline
LightGBM + ELO (35 features)	**0.4521**	**0.4716**	**Canonical, registered**
XGBoost + ELO (35 features)	0.3824	0.5150	Experiment, not registered

4.2 Per-Class F1 Analysis

The canonical LightGBM model achieves per-class F1 scores of $H = 0.562$, $D = 0.296$, and $A = 0.498$. The Draw class is the hardest — both because it has the lowest base rate (≈ 0.25) and because draws are inherently more random in football than decisive outcomes. The XGBoost run, identical in feature space and hyperparameters, collapsed Draw F1 to 0.03 because XGBoost has no built-in equivalent to LightGBM's `class_weight='balanced'`. This empirically validates the LightGBM choice: out-of-the-box class weighting matters more on this dataset than the choice between gradient-boosted tree libraries.

4.3 Feature Importance

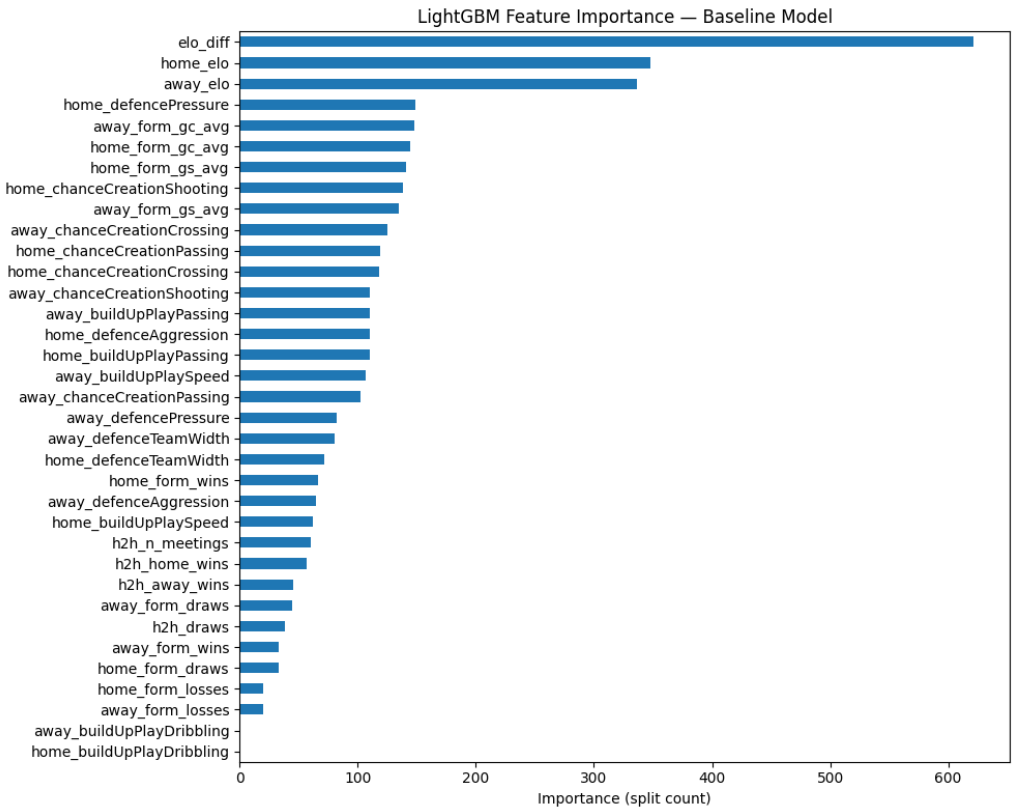

Figure 6: Top-15 feature importances from the canonical LightGBM model. ELO features rank prominently among the top contributors.

4.4 Production Drift Measurement

Table 3: Production drift evaluation: canonical LightGBM v17 evaluated on the held-out 2015/16 season.

Metric	**2014/15 (Test)**	**2015/16 (Production)**
Macro F1	0.4521	0.4379
ROC-AUC	0.6471	0.6359
Accuracy	0.4716	0.4601

The drift in macro F1 is -0.0142 (or -1.4 F1 percentage points). The script's severity classifier assigns this to the MILD DRIFT band: small enough to keep serving the current model, large enough to warrant continued monitoring. This empirically justifies the weekly retraining cadence chosen for the training Airflow DAG.

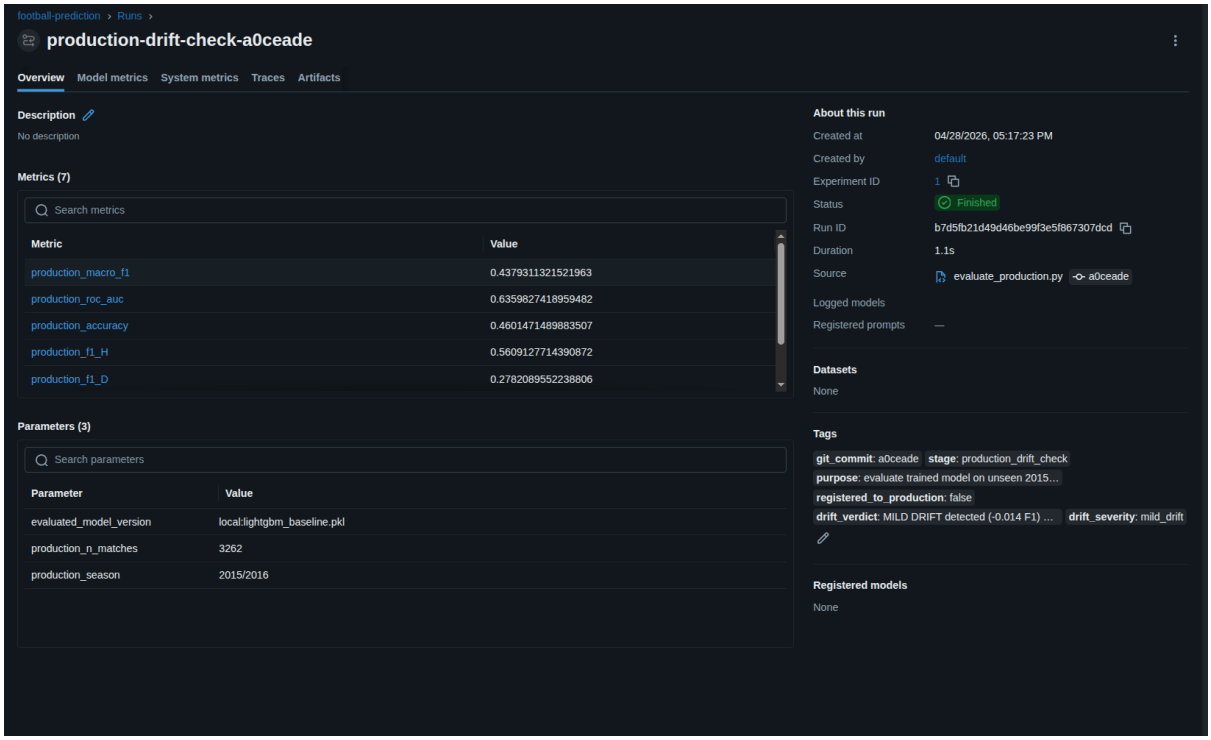

Figure 7: Drift evaluation logged to MLflow as a separate run with custom tags (`drift_severity`, `drift_verdict`, `git_commit`).

```
2026-04-28 17:44:30 | INFO     | __main__ | ============================================================
2026-04-28 17:44:30 | INFO     | __main__ | DRIFT COMPARISON
2026-04-28 17:44:30 | INFO     | __main__ | ============================================================
2026-04-28 17:44:30 | INFO     | __main__ |   Test F1 (2014/15):       0.4521
2026-04-28 17:44:30 | INFO     | __main__ |   Production F1 (2015/16): 0.4379
2026-04-28 17:44:30 | INFO     | __main__ |   Drift (prod - test):     -0.0142
2026-04-28 17:44:30 | INFO     | __main__ |   Verdict: MILD DRIFT detected (-0.014 F1) — monitor over next 2 seasons
```

Figure 8: Terminal output from `evaluate_production.py` showing the DRIFT COMPARISON block and verdict.

5 Operations

5.1 Two Airflow DAGs

The system has two independent DAGs running on different cadences:

- **football_training_pipeline** (weekly, Sundays at 02:00 UTC) — the retraining DAG. Validates trigger file freshness, runs feature engineering and training as DVC stages, registers the new model, gates registration on a quality threshold (Test F1 $\geq$ 0.40).
- **football_drift_check** (daily, 03:00 UTC) — the drift detection DAG. Runs `evaluate_production` as a DVC stage, parses the resulting drift report, and fails the DAG only on `severe_drift` (a >5 F1 percentage point drop). Milder drift passes silently to avoid alert fatigue.

The two-DAG split reflects the fact that drift detection answers a different question than retraining: "is the deployed model still good?" versus "should we update?". In production with streaming data, drift could appear days before the next scheduled retrain — a daily check catches this. Both DAGs share a single-slot `training_pool` so they never run concurrently.

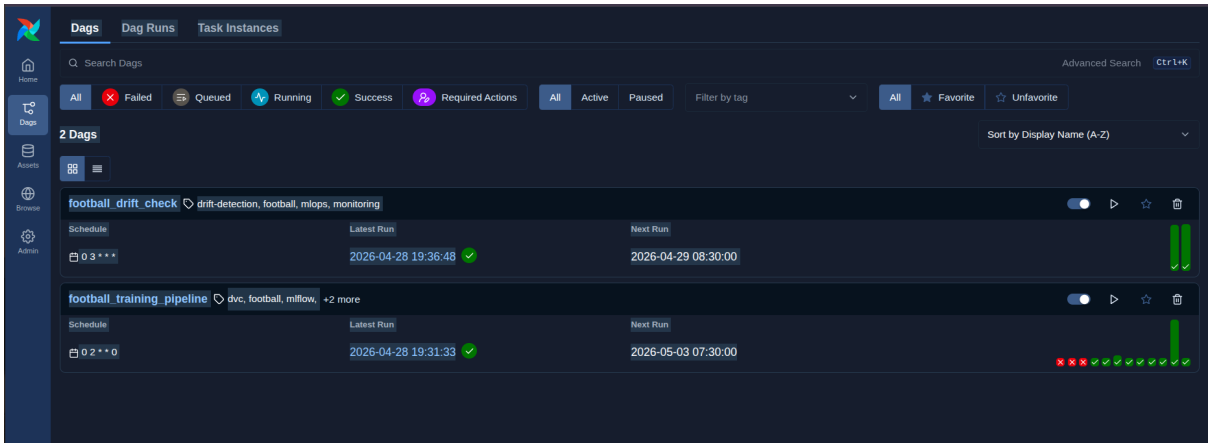

Figure 9: Airflow DAGs list showing both `football_training_pipeline` (weekly) and `football_drift_check` (daily).

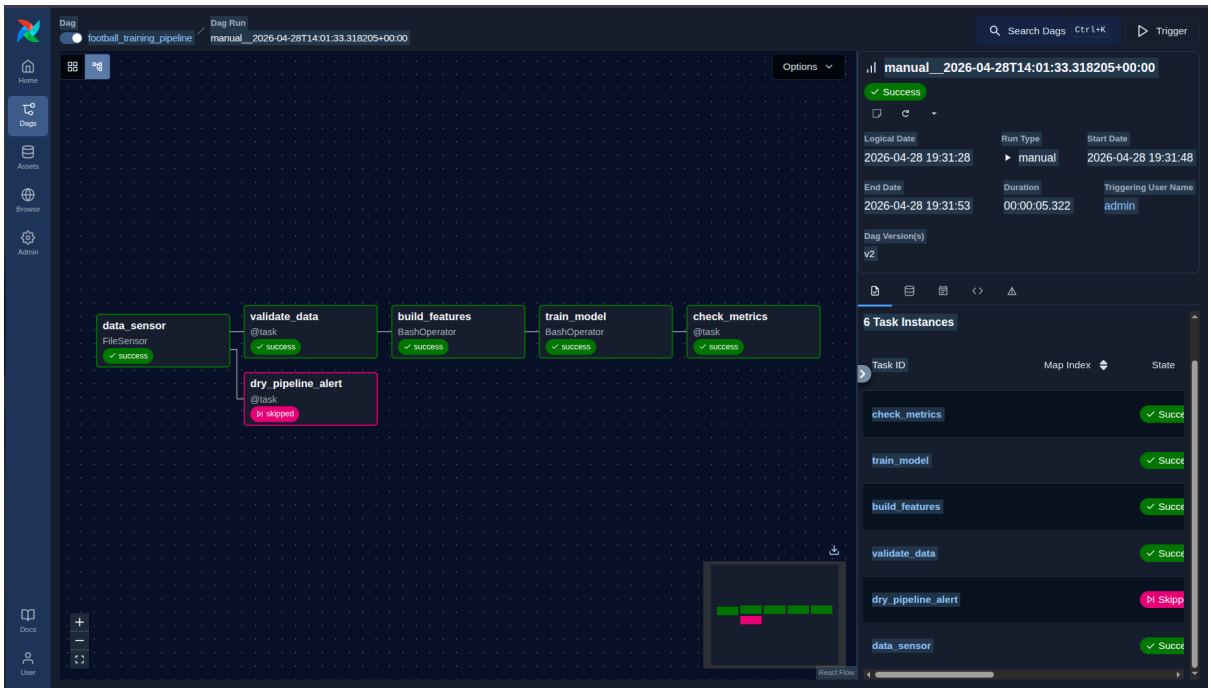

Figure 10: Training pipeline graph view: `data_sensor` → `validate_data` → `build_features` → `train_model` → `check_metrics` chain plus the `dry_pipeline_alert` branch.

5.2 Test Suite

The pytest suite covers 14 test cases across six categories: smoke tests, input validation, cold-start handling, target derivation, data leakage prevention, and a model-input contract test that verifies the runtime feature vector matches the trained model's expectations. Total runtime: 1.4 seconds.

```
project$ # Run pytest with verbose flag
pytest tests/ -v
============================================= test session starts =============================================
platform linux -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- /home/muhammed786fiyas/miniconda3/envs/football-mlops
/bin/python3.11
cachedir: .pytest_cache
rootdir: /home/muhammed786fiyas/Desktop/Projects/ml_ops/final_project/mlops_end_to_end_project
plugins: anyio-4.13.0, hydra-core-1.3.2
collected 14 items

tests/test_api_smoke.py::test_health_endpoint_returns_ok PASSED                                          [  7%]
tests/test_api_smoke.py::test_teams_endpoint_returns_full_lookup PASSED                                  [ 14%]
tests/test_api_smoke.py::test_metrics_endpoint_exposes_prometheus_format PASSED                          [ 21%]
tests/test_api_validation.py::test_unknown_home_team_returns_400 PASSED                                  [ 28%]
tests/test_api_validation.py::test_same_team_for_home_and_away_returns_400 PASSED                        [ 35%]
tests/test_api_validation.py::test_missing_required_field_returns_422 PASSED                             [ 42%]
tests/test_api_validation.py::test_valid_request_returns_prediction PASSED                               [ 50%]
tests/test_build_features.py::test_derive_target_produces_correct_h_d_a PASSED                           [ 57%]
tests/test_build_features.py::test_derive_target_raises_on_null_outcomes PASSED                          [ 64%]
tests/test_build_features.py::test_chronological_split_raises_on_overlapping_seasons PASSED              [ 71%]
tests/test_build_features.py::test_chronological_split_raises_on_unknown_seasons PASSED                  [ 78%]
tests/test_build_features.py::test_chronological_split_clean_split_succeeds PASSED                       [ 85%]
tests/test_feature_service.py::test_cold_start_team_returns_zero_form PASSED                             [ 92%]
tests/test_feature_service.py::test_feature_columns_match_train_contract PASSED                          [100%]

============================================== 14 passed in 1.30s ==============================================
```

Figure 11: Full pytest suite passing: 14 test cases across smoke, validation, contract, and leakage categories.

The contract test deserves particular mention: it imports `FEATURE_COLUMNS` from the training script and `_expected_columns()` from the runtime feature service, then asserts they are identical. This is the test that would have caught any silent train/serve schema drift during the Day 7 ELO refactor; without it, a missing column at inference time would only surface as a runtime exception in production.

5.3 Monitoring and Email Alerting

The Grafana dashboard provides 11 panels following the Four Golden Signals pattern: backend up/down status, request rate by predicted class, latency percentiles (p50/p95/p99), error rate, error breakdown by status code, predicted-class distribution, currently loaded model version, and an active alerts panel.

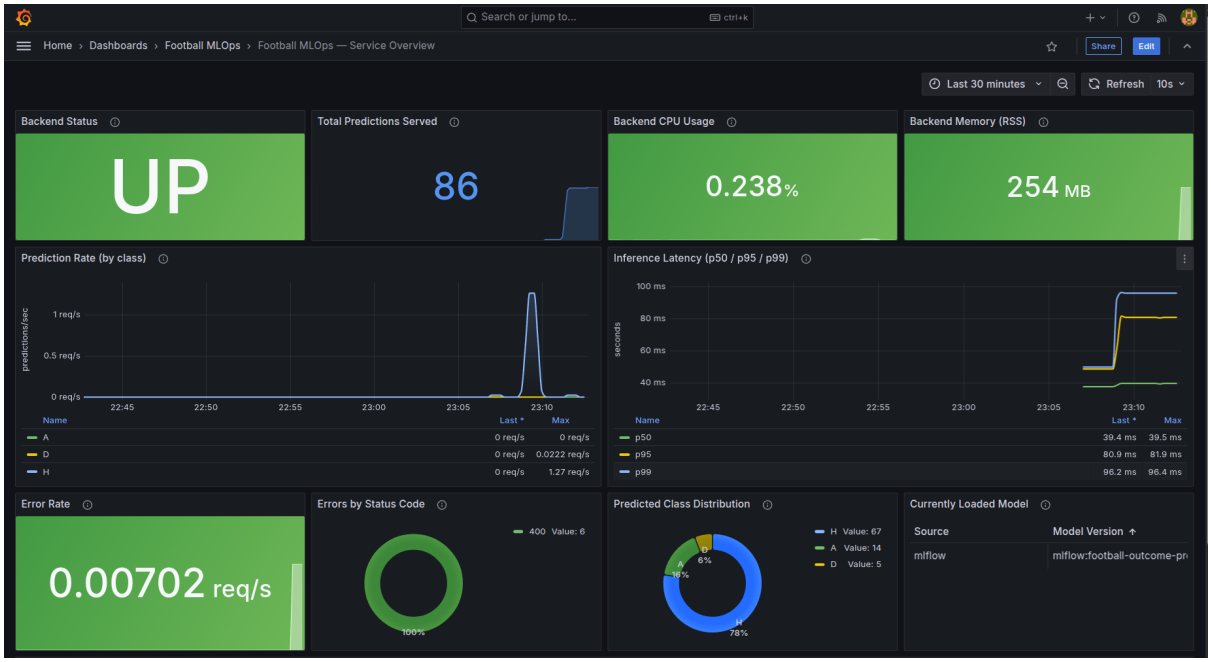

Figure 12: Grafana service overview dashboard with 11 panels following the Four Golden Signals pattern.

Critical-severity alerts (such as `BackendDown`, fired when Prometheus has not scraped the backend for 60 seconds) fan out to two receivers: a webhook-logger for stdout debugging, and a custom Flask-based MailTrap relay that translates AlertManager's webhook payload into a MailTrap REST API email send. This pattern was chosen over native AlertManager SMTP because the webhook approach reads credentials from environment variables (loaded from `.env`) without requiring `envsubst` or template rendering at AlertManager startup.

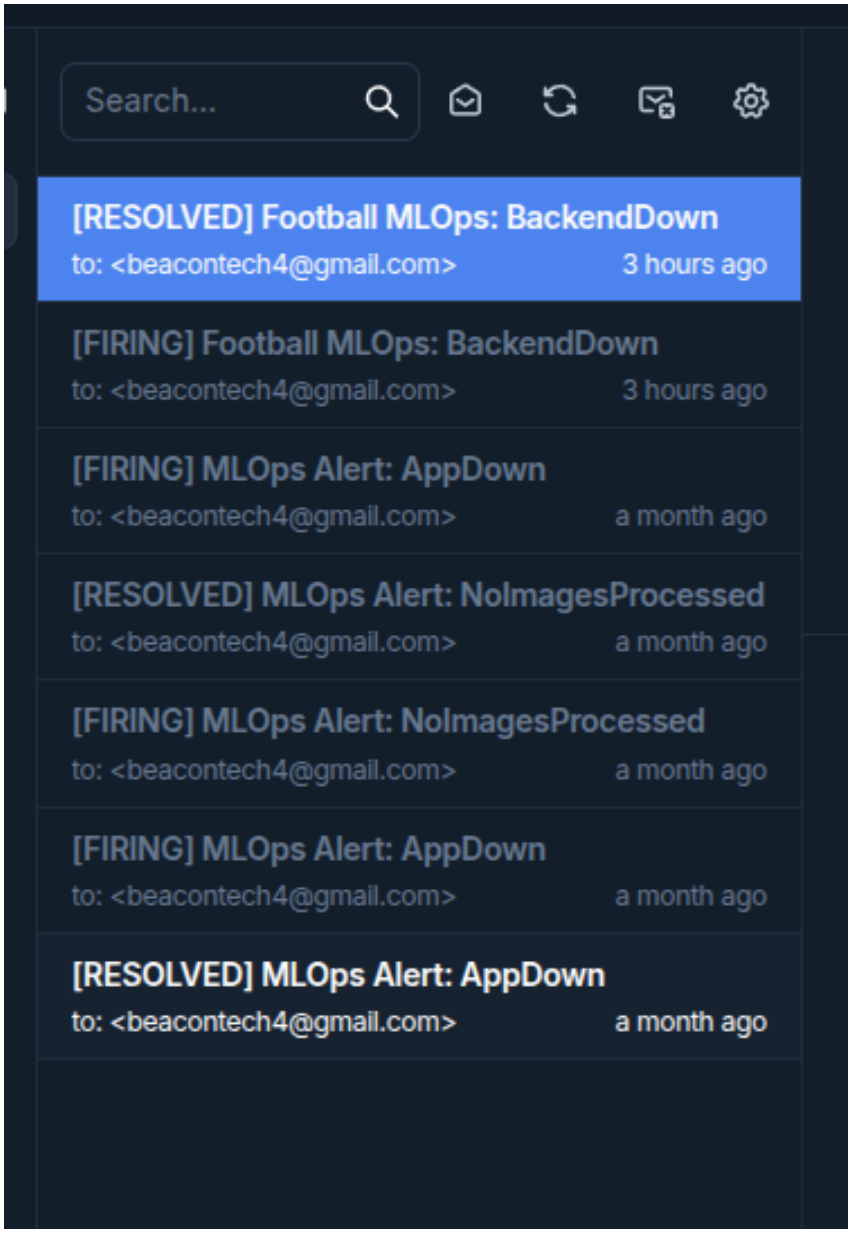

Figure 13: MailTrap sandbox inbox: alert email arrives within 90 seconds of triggering `BackendDown`.

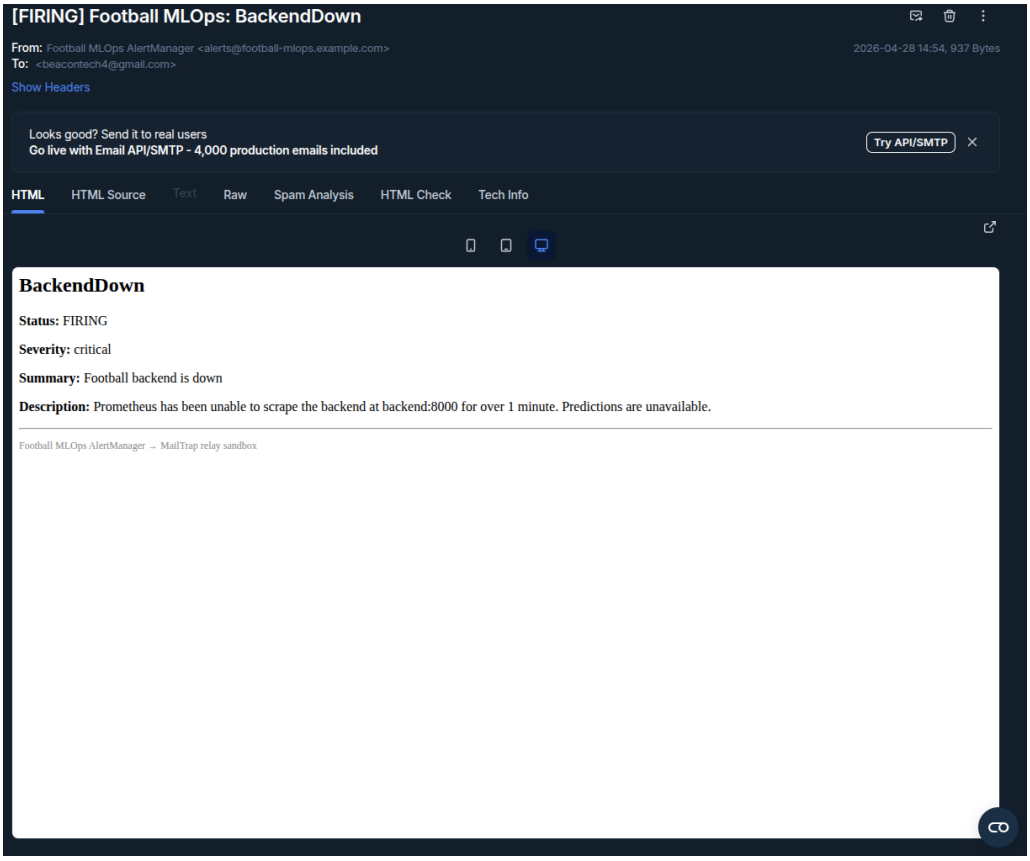

Figure 14: Alert email body with formatted alert details: name, status, severity, summary, description.

5.4 User-Facing Prediction UI

Figure 15: Frontend prediction UI: Real Madrid (home) vs. Granada returns Home Win with ∼74% probability, ∼50 ms latency, and the serving container ID for traceability.

6 Conclusion

This project delivered a 12-container production-shape MLOps system for football match outcome prediction. The model achieves test macro F1 = 0.4521 (ELO-improved canonical) with measured drift of −1.4 F1 percentage points on an unseen season. While this falls short of the ambitious F1 = 0.50 target, the implementation demonstrates the full MLOps lifecycle: reproducible data and model lineage via DVC, experiment tracking with three compared models in MLflow, scheduled retraining and drift detection via two Airflow DAGs, monitoring via Prometheus + Grafana, and real email alerting via a MailTrap relay sidecar.

The most important engineering lessons came from problems faced and mitigated during the build: a contract test caught a Day 7 train/serve schema mismatch before it reached production; a graceful MLflow registry → local pickle fallback kept the backend functional during artifact-mount problems; defensive trigger-file freshness checks in the Airflow DAG correctly fired on a 25-hour-old trigger; and a deliberate two-DAG separation (training vs. drift) anticipated production cadences that a single DAG would conflate.

The full project, with daily build journal, git tags for every milestone, and comprehensive design documentation (HLD, LLD, User Manual, Test Plan), is available at https://github.com/muhammed786fiyas/mlops_end_to_end_project.

References

- Hugo Mathien. "European Soccer Database." Kaggle. https://www.kaggle.com/datasets/hugomathien/soccer
- Apache Airflow Documentation. https://airflow.apache.org/docs/
- DVC Documentation. "Pipelines and Stages." https://dvc.org/doc/start/data-pipelines
- MLflow Documentation. "Model Registry." https://mlflow.org/docs/latest/model-registry.html
- Prometheus Documentation. "Best Practices: Histograms and Summaries." https://prometheus.io/docs/practices/histograms/